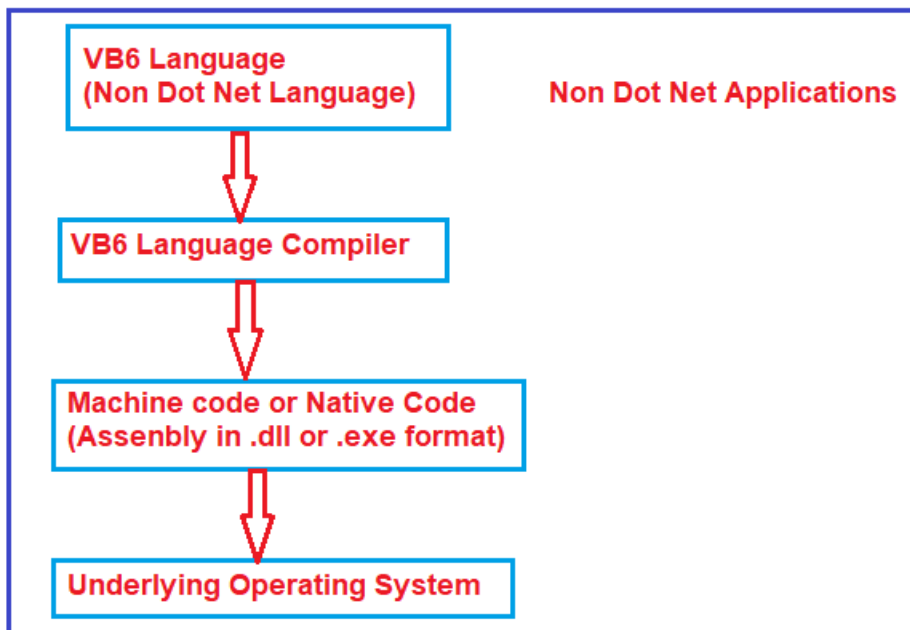


.NET Program Execution Process Flow:

In this article, I will discuss the **.NET Program Execution Process Flow** in Detail. Please read our previous article discussing the [Common Language Runtime \(CLR\)](#) architecture in detail. As .NET Developers, we should know when we create an application, how the application is compiled, and how the .NET Framework executes the application. But before understanding the **.NET Program Execution** process, let us first understand how non-dot net applications such as C, VB6, and C++ programs are executed.

Non-DOT NET Program Execution Process:

We know that computers only understand machine-level code. The Machine-level code is also known as native code or binary code. So when we compile a C, VB6, or C++ program, the language compiler compiles the language source code. It generates the native machine code (also called binary code), which can be understood by the underlying operating system and the system hardware. The above process is shown in the below image.



The Native code or machine code that is generated by the respective language compiler is specific to the operating system on which it is generated. If we take this compiled native code and try to run it on another operating system, then it will fail. So, the problem with this style of program execution is that it is not portable from one platform to another. That means it is platform-dependent.

.NET Program Execution Process:

Now, let us understand the .NET Program Execution Process in detail. Using .NET, we can create different types of applications such as Console, Windows, Web, and Mobile Applications. Irrespective of the type of application, when we execute any .NET application, the following things happen in order

The .NET application Source Code gets compiled into Microsoft Intermediate Language (MSIL), which is also called Intermediate language (IL) or Common Intermediate language (CIL) code. Both .NET and Non-DOTNET applications generate an assembly when we compile the application. Generally, the assemblies have an extension of .DLL or .EXE based on the type of application we compiled. For example, if we compile

a Window or Console application in .NET, we get an assembly of type .EXE, whereas when we compile a Web or Class Library Project in .NET, we get an assembly of type .DLL.

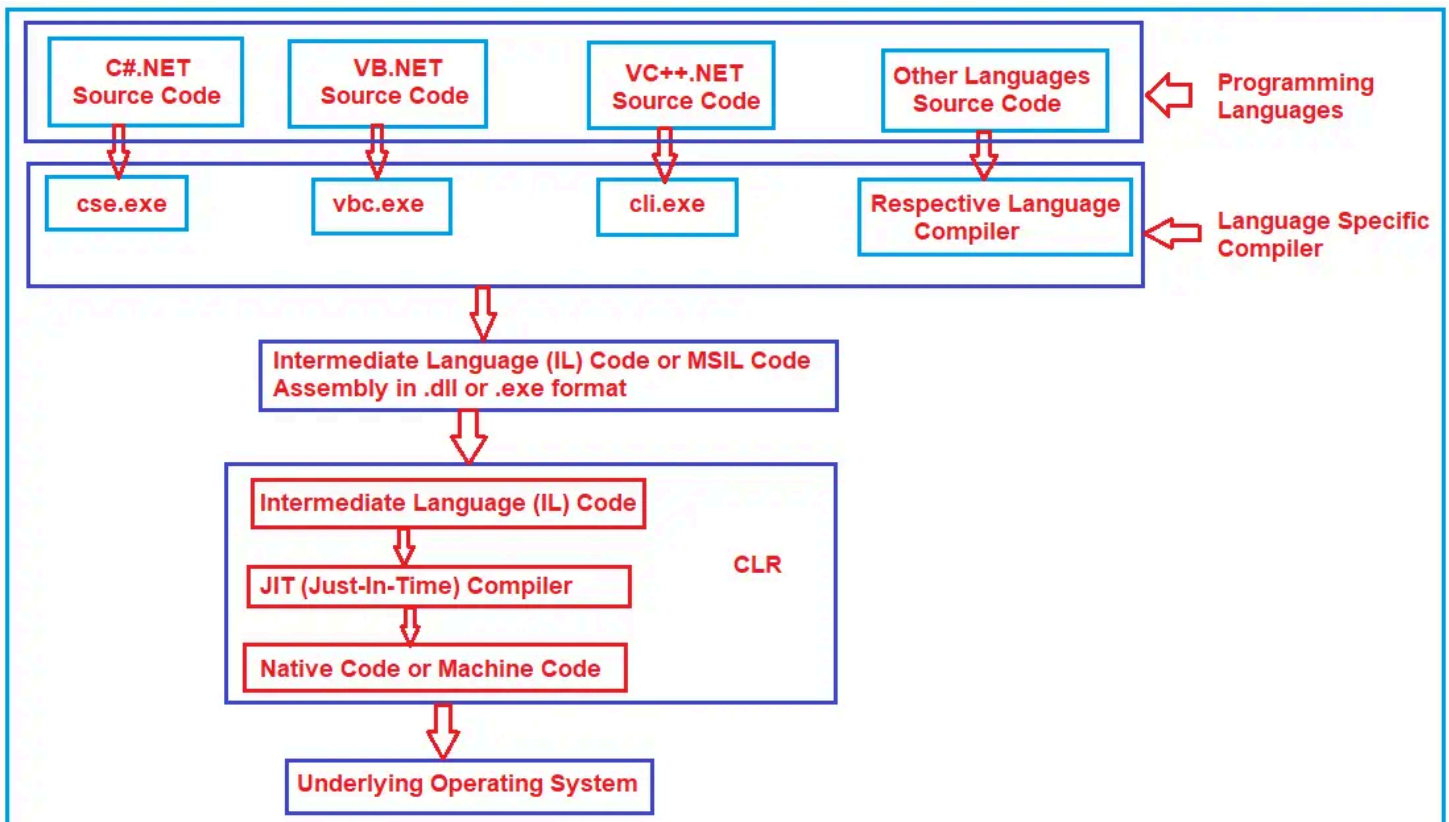
The difference between a .NET and NON-DOTNET assembly is that .NET Assembly is an Intermediate Language format, whereas NON-.NET assembly is in native code format.

The NON .NET applications can run directly on top of the operating system as the NON-DOTNET assembly contains the native code, whereas .NET applications run on top of a virtual environment called **Common Language Runtime (CLR)**. CLR contains a component called **Just-In-Time Compiler (JIT)** which will convert the Intermediate language into native code, which can be understood by the underlying operating system.

.NET Program Execution Steps:

In .NET, the application execution consists of 2 steps. They are as follows:

In the step1 the respective language compiler compiles the Source Code into Intermediate Language (IL). In the 2nd step, the JIT compiler of CLR will convert the Intermediate Language (IL) code into native code (Machine Code or Binary Code), which the underlying operating system can then execute. The above process is shown in the image below.



As the .NET assembly is in Intermediate Language (IL) format and not in native or machine code, the .NET assemblies are portable to any platform as long as the target platform has the **Common Language Runtime (CLR)**. The target platform's CLR converts the Intermediate Language code into native or machine code that the underlying operating system can understand.

Intermediate Language code is also called managed code. This is because CLR manages the code that runs inside it. For example, in a VB6 program, the developer is responsible for de-allocating the memory

consumed by an object. If a programmer forgets to de-allocate memory, it may get out-of-memory exceptions. On the other hand, a .NET programmer does not need to worry about de-allocating the memory consumed by an object. Automatic memory management also known as garbage collection is provided by CLR. Apart from garbage collection, the CLR provides several other benefits, which we will discuss in a later session. Since CLR manages and executes the Intermediate Language, it (IL) is also called the managed code.

.NET supports different programming languages like C#, VB, J#, and C++. C#, VB, and J# can only generate managed code (IL), whereas C++ can generate both managed code (IL) and unmanaged code (Native code).

The native code is not stored permanently anywhere after we close the program the native code is thrown away. When we execute the program again, the native code is generated again.

The .NET program is similar to the Java program's execution. In Java, we have bytecodes and JVM (Java Virtual Machine) whereas in .NET we have Intermediate Language and CLR (Common Language Runtime).

In the next article, I will discuss the **Intermediate Language (IL Code) in C#** with examples. In this article, I try to explain the DOT NET Program Execution process. I hope you enjoy this article.